

Lecture 12

In React, **props** (short for properties) are the primary mechanism for passing data from a parent component to a child component. They are essential for making components reusable and dynamic.

+1

1. Core Characteristics

- **Unidirectional Data Flow:** Props always flow **downwards** (Parent=>child)
- **Read-Only (Immutable):** A component must never modify its own props. They are treated as "pure" inputs.
- **Plain JavaScript Objects:** Under the hood, props are just an object containing the attributes passed from the JSX.

2. Syntax & Implementation

Passing Props

Attributes are added to the component tag just like HTML attributes.

JavaScript

```
<UserProfile name="Alice" age={25} isAdmin={true} />
```

Receiving Props

In the child component, props are received as the first argument to the function.

JavaScript

```
// Option 1: Using the props object
function UserProfile(props) {
  return <h1>Hello, {props.name}</h1>;
}
```

```
// Option 2: Destructuring (Most Common)
function UserProfile({ name, age }) {
  return (
    <div>
```

```
    <p>Name: {name}</p>
    <p>Age: {age}</p>
  </div>
);
}
```

3. What can be passed as Props?

You aren't limited to strings or numbers. You can pass:

- **Primitive types:** string, number, boolean.
- **Objects and Arrays:** user={{id: 1}}, tags={['react', 'web']}.
- **Functions:** Used for **Lifting State Up** (allowing a child to communicate back to the parent).
- **React Components/JSX:** Passing UI elements as data.

4. Special Prop: children

The `children` prop is automatically passed to every component. It represents the content between the opening and closing tags of a component.

```
+1
JavaScript

function Card({ children }) {
  return <div className="card-style">{children}</div>;
}

// Usage
<Card>
  <p>This is inside the card!</p>
</Card>
```

5. Comparison: Props vs. State

Feature	Props	State
Origin	Received from parent	Managed within the component
Mutability	Immutable (Read-only)	Mutable (via <code>setState</code> or <code>useState</code>)
Purpose	Configuration/Data passing	Managing internal interactivity

Here are **all common datatypes** you can pass as props:

1. String

```
<Student name="Ganesh" />
```

2. Number

```
<Student age={21} />
```

3. Boolean

```
<Student isActive={true} />
```

Shortcut:

```
<Student isActive />
```

4. Array

```
<Student skills={["React", "Node", "MongoDB"]} />
```

5. Object

```
<Student details={{ city: "Chennai", course: "React" }} />
```

✓ 6. Function

```
<Student handleClick={() => alert("Hello")} />
```

✓ 7. JSX Element

```
<Student icon={<h1>🔥 </h1>} />
```

✓ 8. Component as Prop

```
<Student profileComponent={<Profile />} />
```

✓ 9. Null

```
<Student data={null} />
```

✓ 10. Undefined

```
<Student data={undefined} />
```

✓ 11. Date Object

```
<Student joinedDate={new Date()} />
```

✓ 12. Symbol (Rare but possible)

```
<Student uniqueId={Symbol("id")} />
```

✓ Full Example (All in One)

Parent Component

```
import Student from "./Student";

function App() {
  const studentObj = { city: "Hyderabad", course: "React" };

  const showMessage = () => {
    alert("Props function called!");
  };

  return (
    <div>
      <Student
        name="Ganesh"           // string
        age={22}                // number
        isActive={true}         // boolean
        skills=["React", "Node"] // array
        details={studentObj}     // object
        handleClick={showMessage} // function
        icon={<h2>🔥</h2>}       // JSX element
        joinedDate={new Date()}  // date object
        data={null}              // null
        value={undefined}        // undefined
      />
    </div>
  );
}

export default App;
```

Child Component

```
function Student(props) {
  return (
    <div>
      <h1>{props.name}</h1>
      <p>Age: {props.age}</p>
      <p>Active: {props.isActive ? "Yes" : "No"}</p>

      <p>Skills: {props.skills.join(", ")}</p>

      <p>City: {props.details.city}</p>
      <p>Course: {props.details.course}</p>

      <p>Joined: {props.joinedDate.toDateString()}</p>

      {props.icon}

      <button onClick={props.handleClick}>Click Me</button>

      <p>Data: {String(props.data)}</p>
      <p>Value: {String(props.value)}</p>
    </div>
  );
}

export default Student;
```
